TRIPLE-A · ENGINEERING HIRING

Stablecoin Checkout

Build the hosted payment page a shopper lands on after they choose to pay with crypto. A quote with a fixed rate and a clock, an address to send money to, and **a result that arrives from somewhere you do not control.**

ROLE	TIME	SEND US
Frontend Engineer	4 to 6 hours We mean it	Git repository with full history, README, design doc, ADRs, agent instructions file

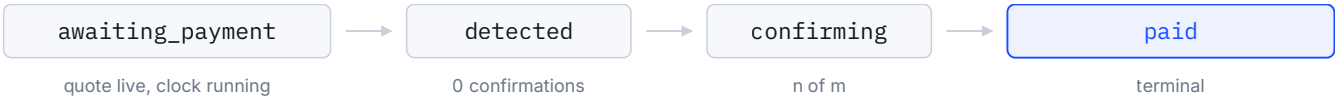
CONTEXT

A shopper picks "Pay with crypto" at checkout. They land on a hosted payment page. The page shows a crypto amount at a fixed rate, the rate expires after some time, and it shows an address to send the money to. Then the page waits. The confirmation comes from a blockchain, not from a form submit.

That waiting part is the interesting part of the job. This exercise is a smaller version of it.

Build only the page the shopper sees. No auth, no real chains, no merchant dashboard.

PAYMENT LIFECYCLE



There are six more states. You can find them in the API section. Work out what each one means for the shopper.

TIME

Please spend 4 to 6 hours. We will not give extra points to someone who worked the whole weekend. If something is missing, that is fine, just write in the README what you dropped and why. We care more about what you built first than about how much you finished.

If you use coding agents, you will go faster than this. Please use the extra time to think, not to add more features.

WHAT TO BUILD

Summary	Merchant name, order reference, and the fiat amount to pay (EUR 149.90), formatted for the shopper's locale.
Currency and network	USDT (Tron, Ethereum), USDC (Ethereum, Polygon, Solana), ETH (Ethereum). Every combination has a different rate, fee and address. When the shopper changes it, you fetch a new quote. If a shopper sends on the wrong network, the money is lost. So make the network impossible to miss.
Quote	Show the exact crypto amount, the address, a QR code, a copy button, and a countdown to `expires_at`. The countdown must still be correct after the tab was in the background for two minutes. When it reaches zero, show an expired state and a way to get a new quote.
Waiting	Poll `GET /api/payments/:reference`. The status `detected` means zero confirmations. Tell the shopper the money arrived, and stop the quote from expiring. A payment that is already on the way must never expire under the shopper. Stop polling on terminal states. No leaked intervals, no overlapping requests.
Edge cases	The happy path is the easy part. In real life the money arrives short, arrives late, arrives twice, or never arrives, and sometimes the server is down. The API gives you the states for all of this. Work out which ones the shopper can still fix, and show those in a different way from the ones they cannot fix. Explain your thinking in the design doc.

THE UI IS YOUR DECISION

We told you what information the page must show. We did not tell you how it should look, or how the flow should work. Layout, visual style, how many steps, what is visible at once and what comes later. All of that is open.

> **If you think one part of the flow should work in a different way, build your version and explain why in the design doc. If your idea makes sense, that is good for you.**

We want to see your judgement about what a shopper needs to see when they send money to an address.

Out of scope: auth, real wallets, persistence, pixel perfect design, real translations.

MOCK API

Serve these fixtures in any way you like. MSW, json-server, thirty lines of Express. Commit the mock. All money amounts are decimal strings.

GET /api/currencies

CODE	DECIMALS	NETWORK ID	NAME	NETWORK FEE	CONFIRMATIONS	AVG SECONDS
USDT	6	tron	Tron (TRC-20)	1.00	1	60
		ethereum	Ethereum (ERC-20)	4.50	3	180
USDC	6	ethereum	Ethereum (ERC-20)	4.50	3	180
		polygon	Polygon	0.10	6	30
		solana	Solana	0.01	1	15
ETH	18	ethereum	Ethereum	3.20	3	180

RESPONSE 200 — SHAPE, ONE ENTRY SHOWN

```
{
  "currencies": [
    {
      "code": "USDT",
      "name": "Tether",
      "decimals": 6,
      "networks": [
        {
          "id": "tron",
          "name": "Tron (TRC-20)",
          "network_fee": "1.00",
          "required_confirmations": 1,
          "avg_confirmation_seconds": 60
        }
      ]
    }
  ]
}
```

POST /api/payments

REQUEST

```
{ "order_id": "ORD-88213", "currency": "USDT", "network": "tron" }
```

RESPONSE 201


```
{
  "payment_reference": "AQH-100306-PMT",
  "order_id": "ORD-88213",
  "status": "awaiting_payment",
  "merchant": { "name": "Nordwind Audio", "logo_url": null },
  "order": { "currency": "EUR", "amount": "149.90" },
  "quote": {
    "crypto_currency": "USDT",
    "network": "tron",
    "network_name": "Tron (TRC-20)",
    "exchange_rate": "0.9214",
    "crypto_amount": "162.69",
    "network_fee": "1.00",
    "total_due": "163.69",
    "crypto_address": "TQ5Nn8kLpVv3xJ7wYcR2bF9aH4dM6sGz1e",
    "required_confirmations": 1,
    "expires_at": "2026-08-14T08:52:10.842Z"
  }
}
```


GET /api/payments/:reference

Returns the payment with an updated `status`, plus the fields below. One fixture per state.

AWAITING_PAYMENT


```
{
  "payment_reference": "AQH-100306-PMT",
  "status": "awaiting_payment"
}
```


DETECTED


```
{
  "payment_reference": "AQH-100306-PMT",
  "status": "detected",
  "confirmations": 0,
  "required_confirmations": 1,
  "amount_received": "163.69",
  "tx_hash": "9d1f4c8a2be7...",
  "detected_at": "2026-08-14T08:44:02.120Z"
}
```


CONFIRMING


```
{
  "payment_reference": "AQH-100306-PMT",
  "status": "confirming",
  "confirmations": 2,
  "required_confirmations": 3,
  "amount_received": "163.69",
  "tx_hash": "9d1f4c8a2be7..."
}
```


PAID


```
{
  "payment_reference": "AQH-100306-PMT",
  "status": "paid",
  "confirmations": 3,
  "required_confirmations": 3,
  "amount_received": "163.69",
  "tx_hash": "9d1f4c8a2be7...",
  "settled_at": "2026-08-14T08:47:31.004Z"
}
```


UNDERPAID


```
{
  "payment_reference": "AQH-100306-PMT",
  "status": "underpaid",
  "amount_received": "120.00",
  "amount_outstanding": "43.69",
  "crypto_address":
"TQ5Nn8kLpVv3xJ7wYcR2bF9aH4dM6sGz1e",
  "tx_hash": "9d1f4c8a2be7..."
}
```


OVERPAID


```
{
  "payment_reference": "AQH-100306-PMT",
  "status": "overpaid",
  "amount_received": "180.00",
  "amount_excess": "16.31",
  "tx_hash": "9d1f4c8a2be7...",
  "settled_at": "2026-08-14T08:47:31.004Z"
}
```

EXPIRED

```
{
  "payment_reference": "AQH-100306-PMT",
  "status": "expired",
  "expired_at": "2026-08-14T08:52:10.842Z"
}
```

FAILED

```
{
  "payment_reference": "AQH-100306-PMT",
  "status": "failed",
  "reason": "settlement_rejected"
}
```

`POST /api/payments/:reference/requote`

Returns 201 with the same body as `POST /api/payments`. Same reference, new quote, status back to `awaiting_payment`.

REQUEST

```
{ "currency": "USDT", "network": "tron" }
```

ERROR — APPLICATION/PROBLEM+JSON

```
{
  "type": "https://developers.triple-a.io/errors/quote-not-expired",
  "title": "Quote has not expired",
  "status": 409,
  "detail": "The current quote is valid until 2026-08-14T08:52:10.842Z."
}
```

WE NEED TO DRIVE THE MOCK

We must be able to trigger any of these states when we want, plus a network error and a slow response. Query param, dev panel, anything you like. Write in the README how to do it.

DOCUMENTATION

Two written documents come with the code. We read them with the same attention as the code. A working page with weak documentation will not pass. We want to see how you think about a problem and how you explain it.

Design doc

What you built and why. Please cover:

- The payment lifecycle state model
- Component structure and data flow
- How the countdown stays correct when the tab is in the background
- How you handle decimal precision
- How you show failure states to the shopper

Diagrams are welcome. A state chart of the lifecycle helps a lot.

One thing I would defend. Please finish the design doc with this section. Pick one place where you did something different from this brief, or different from the obvious solution, and explain why. One paragraph is enough. If you built the obvious version everywhere, write that, and tell us which part you would change first with more time.

ADRs

One short architecture decision record for each important choice. Normal format: context, options, decision, consequences. We expect a few, not twelve. Deciding what is important enough for an ADR is part of the exercise.

README

How to run it. How to trigger each payment state. Plus the two sections below.

What I dropped. Anything you cut because of time, and why you cut that thing and not another one.

Working with the agent. Our team works with coding agents every day, so we want to see how you use one. Please tell us:

- Three things your agent wrote that you threw away, and why
- One place where you did it your own way and the code got better
- Anything the agent got wrong about the payment state machine

If you wrote everything by hand, skip this section. Just say so, and we will read the repository in the same way.

COMMIT HISTORY AND AGENT INSTRUCTIONS

Push the real commit history. Please do not squash everything into one commit. We look at how the work happened, including the wrong turns and the reverts. One clean commit tells us nothing.

Commit your agent instructions file too, whatever your tool uses (`CLAUDE.md`, `AGENTS.md`, `.cursorrules`). The rules you give an agent show us what you think good code is.

WHAT HAPPENS NEXT

If this exercise goes well, we invite you to a 45 minute call with two of our engineers. You share your screen and you extend your own code while we watch, with your normal tools. We will add one new requirement during the call.

> We take the decision in that call. So build something you can still explain one week later.

HOW WE SCORE IT

- **Shopper safety.** Is the network impossible to miss? Can a shopper send on the wrong chain by mistake? Does the `overpaid` screen tell them the truth?
- **What the shopper can fix.** Does your UI separate the states the shopper can still act on from the ones they cannot?
- **Correct when things go wrong.** Clock differences, background tabs, no leaked intervals, no overlapping polls, good behaviour when the server returns 500.
- **Decimal precision.** Six decimals for USDT, eighteen for ETH. No floats.
- **Judgement in writing.** Do the ADRs show a real trade-off, or only describe the obvious choice?
- **How you worked with the agent.** Commit history, what you rejected, the instructions file.

> A simple page that gets these right is better than a beautiful page that expires the quote while the shopper's money is on the way.

Triple-A · Questions about this exercise are welcome. Send them to your recruiting contact, before or during.